
Откриване на обекти

Локализация на обекти

- Класификация на изображение



Car

- Класификация с локализация



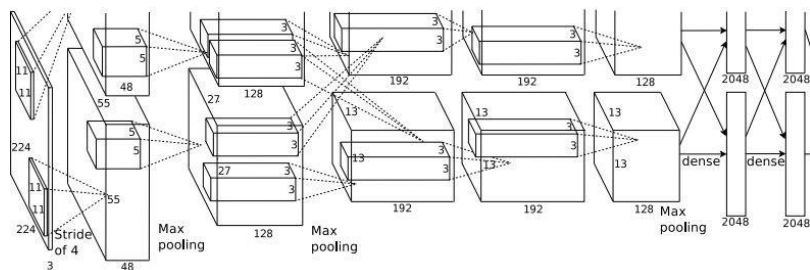
1 Object Car

- Откриване



Multiple objects

Класификация + Локализация



Често предварително
обучен в ImageNet
(трансферно обучение)

Fully
Connected:
4096 to
1000

Class Scores

Cat: 0.9
Dog: 0.05
Car: 0.01
...

Correct label:

Cat

Softmax
Loss

Multitask Loss

+

Loss

Vector:
4096

Fully
Connected:
4096 to 4

Box Coordinates

(x, y, w, h)

L2 Loss

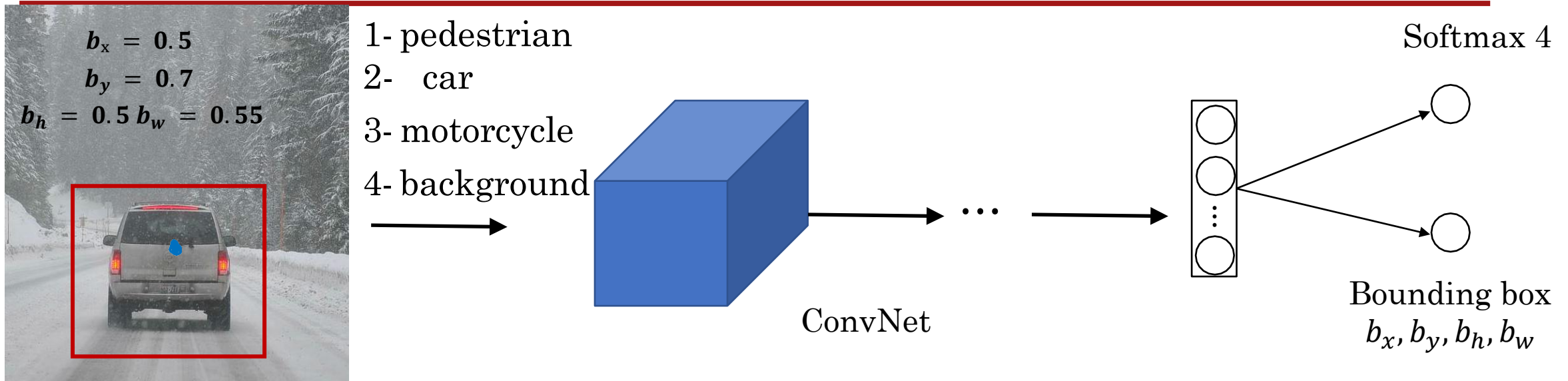
Correct box:
(x', y', w', h')

Локализацията е регресия

Локализация на обекти като мултитаск обучение

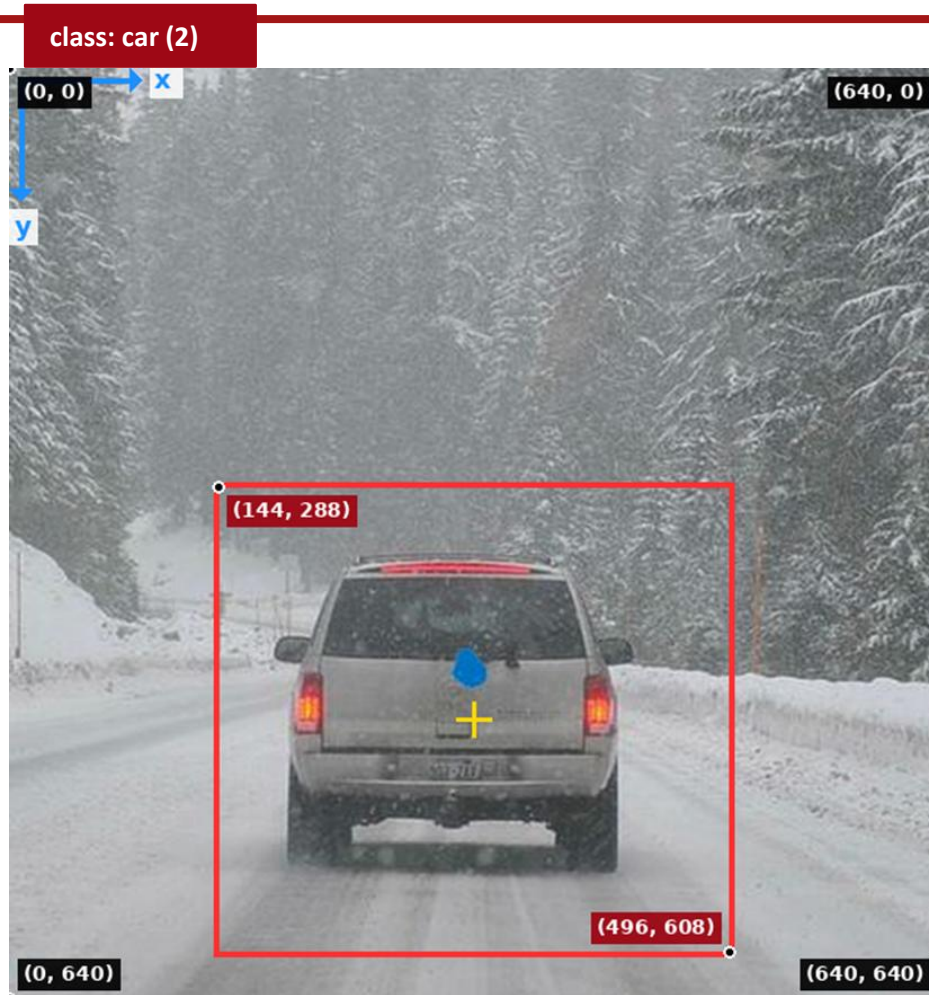
- Конволюционна невронна мрежа (CNN) извлича характеристики от изображението
- Моделът има два паралелни изхода:
 - Класификацията предсказва класа на обекта (напр. котка, куче)
 - Локализацията предсказва координатите на ограничаващата рамка (x, y, w, h)
- Локализацията се разглежда като регресионна задача
- Моделът се обучава чрез комбинирана функция на загуба:
 - Softmax загуба за класификация
 - L2 (IoU) загуба за координатите
- Целта е да се определи едновременно *какво* е обектът и *къде* се намира

Класификация с локализация



- Всички координати на bbox са нормализирани в $[0, 1]$ спрямо размера на изображението.
- Конвенция за bounding box:
- (b_x, b_y) — координати на центъра на обекта нормализирани
- (b_h, b_w) — височина на кутията (като дял от височината на изображението)
— ширина на кутията (като дял от широчината на изображението)

Етикетиране



Изображение 640×640 px · bounding box обхваща колата

Съдържание на label файла

2 0.5 0.7 0.55 0.5

class_id *b_x* *b_y* *b_w* *b_h*

Декодиране в пиксели (W = H = 640)

център x	0.5×640	→ 320 px
център y	0.7×640	→ 448 px
ширина	0.55×640	→ 352 px
височина	0.5×640	→ 320 px

Рамка

горен ляв: (144, 288)

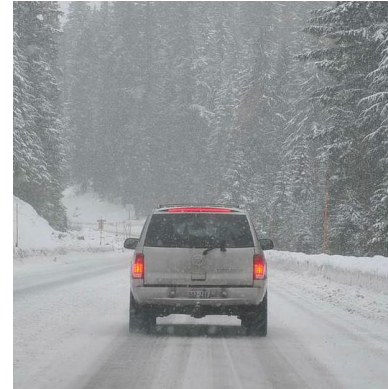
долен десен: (496, 608)

Същият етикет (0.5, 0.7, 0.55, 0.5) работи при всяка резолюция — пикселите се преизчисляват автоматично.

Определяне на целевия етикет y

- 1 - pedestrian
- 2 - car
- 3 - motorcycle
- background

Трябва да се зададе b_x, b_y, b_h, b_w , class label (1-4)



$$Y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Изображение без обект (само фон):
 $y = [0, ?, ?, ?, ?, ?, ?, ?]^T$

Откриване на обекти

Откриване на обекти

Класически подход: Sliding Window

- За sliding window е необходим бинарен класификатор "кола / не кола" върху плътно изрязани изображения.

у



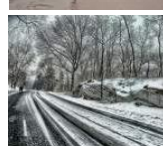
1



1



1



0

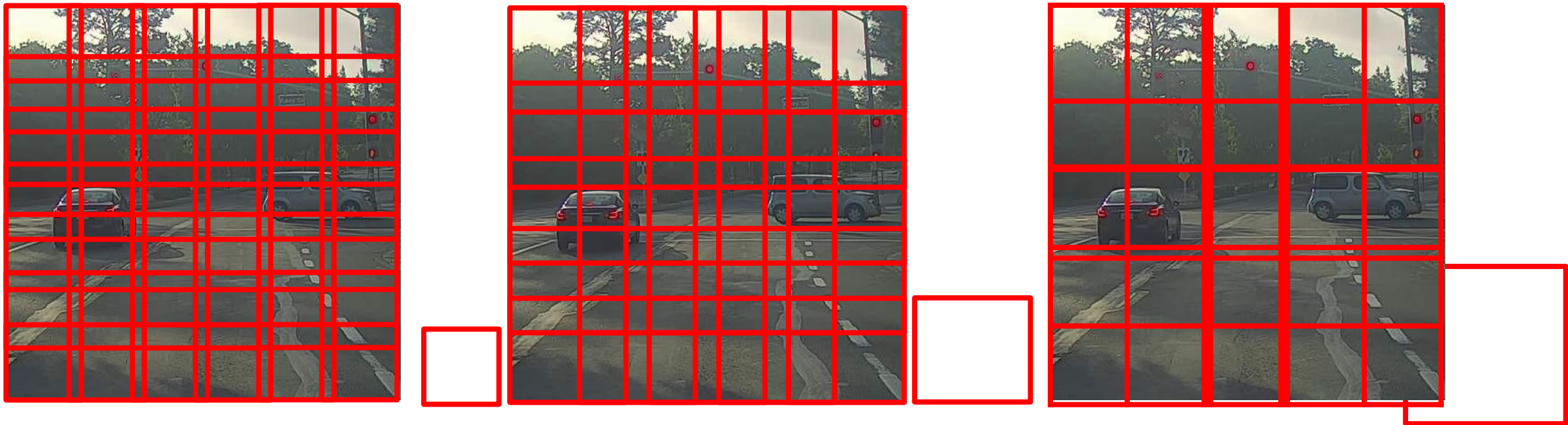


0



Класически подход: Sliding Window

- малък прозорец → среден → голям

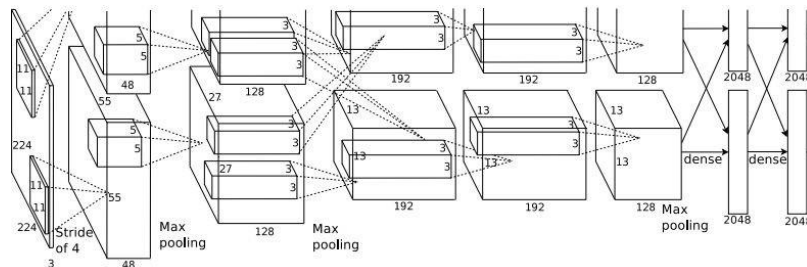


ConvNet на едно изображение се обработва многократно → не е приложимо за работа в реално време

Откриване на обекти

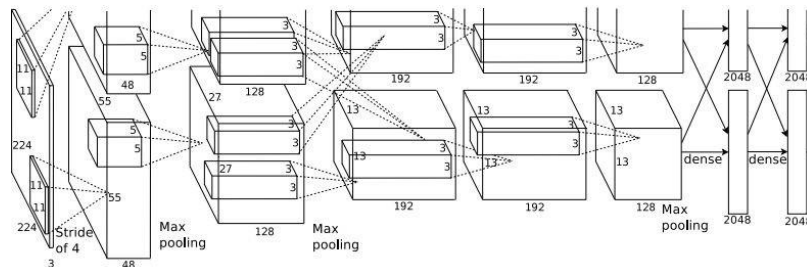
Конволюционно изпълнение на плъзгащи се
прозорци

Откриването на обекти и регресия



CAT: (x, y, w, h)

4 numbers

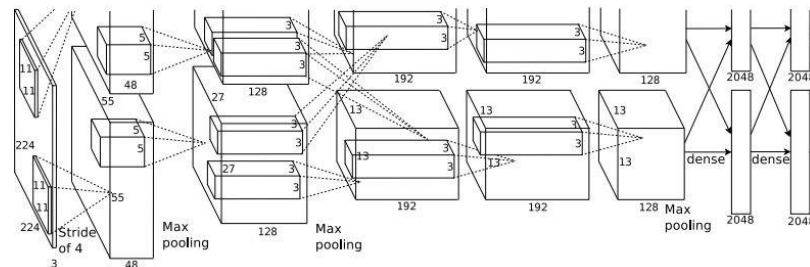


DOG: (x, y, w, h)

DOG: (x, y, w, h)

CAT: (x, y, w, h)

16 numbers

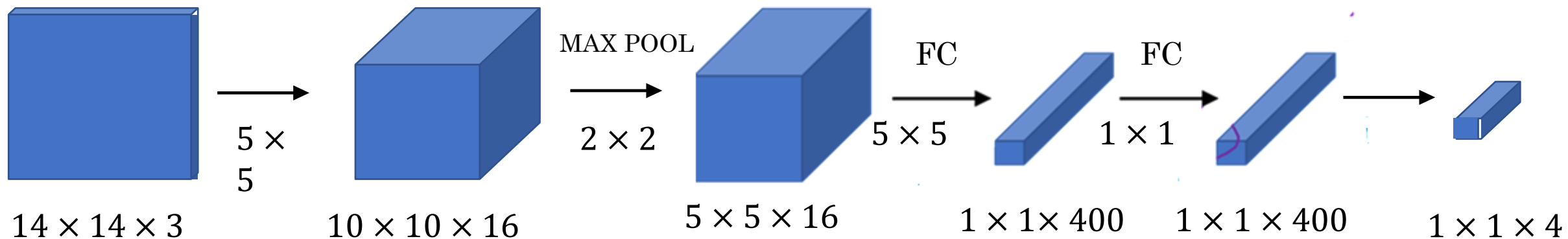
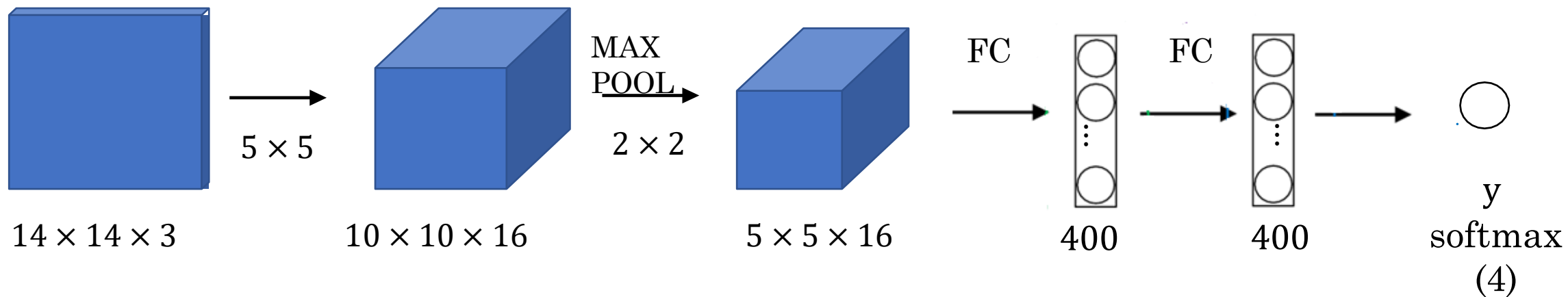


DUCK: (x, y, w, h)

DUCK: (x, y, w, h)

Many
numbers!

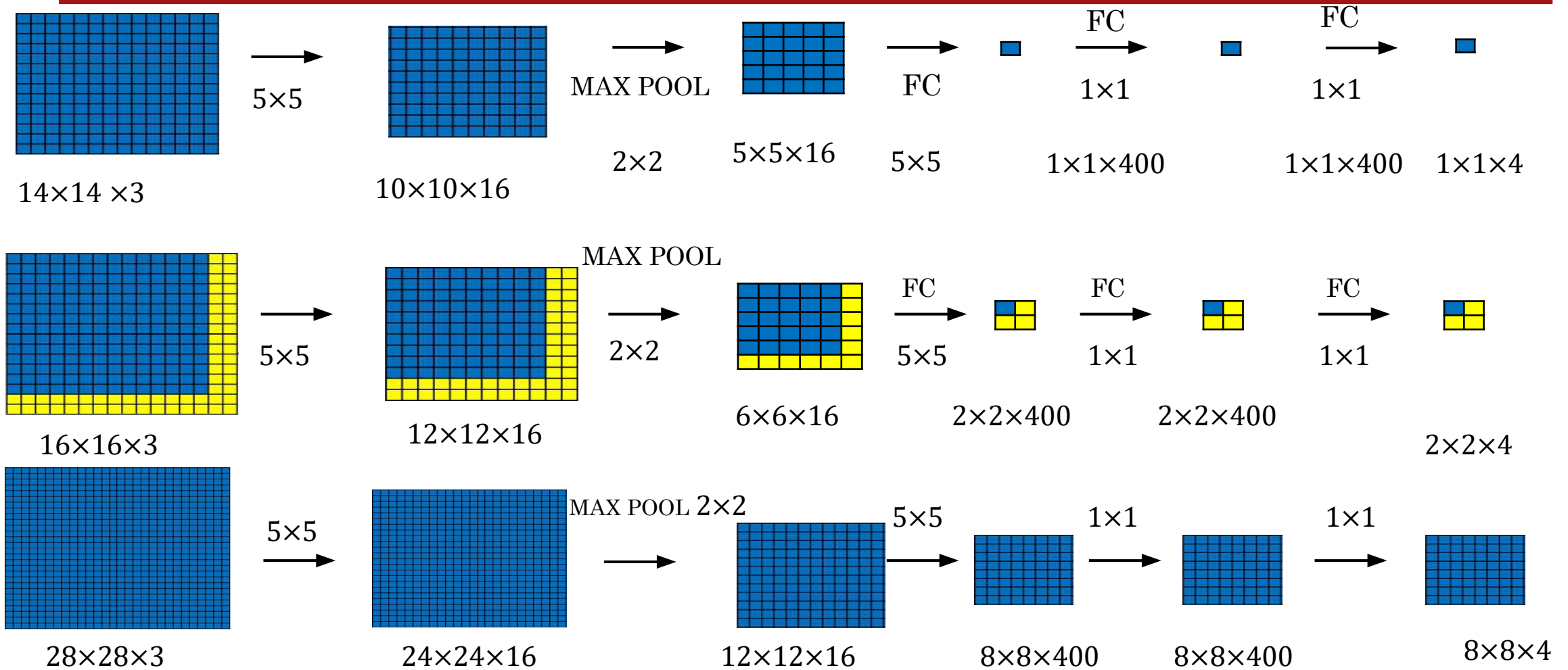
Превръщане на FC слой в конволюционни слоеве



Замяната на FC с конволюция

- Не намалява задължително параметрите
- Позволява обработка на изображения с произволен размер
- Запазва пространствената структура и прави възможна ефективната детекция (напр. в YOLO)

Реализация на плъзгащ прозорец с КОНВОЛЮЦИЯ



Sermanet et al., 2014, OverFeat: Integrated recognition, localization and detection using convolutional networks]

Конволюционна реализация на плъзгащи се прозорци

- Класически подход- за да приложим метода на плъзгащи се прозорци, изрязваме регион от изображението (например 14×14 пиксела), подаваме на мрежата и повтаряме процеса за всеки следващ регион, докато обходим цялото изображение и разпознаем търсения обект (напр. кола)
- Подобрение - с конволюционната реализация не е нужно прозорците да се обработват последователно. Вместо това цялото изображение се подава директно към ConvNet, която прогнозира всички позиции едновременно в едно преминаване напред. Това прави процеса значително по-ефективен

Откриване на обекти

Прогнозиране на ограничителната рамка

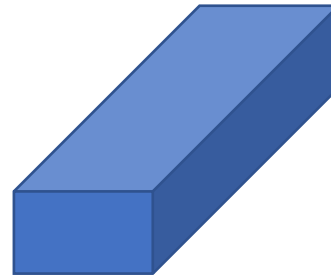
YOLO

- YOLO (You Only Look Once) е модел за разпознаване на обекти в реално време, който:
 - обработва изображението с едно преминаване
 - едновременно предсказва:
 - рамки (bounding boxes)
 - класове на обекти
- Основното му предимство е балансът между скорост и точност

Развитие на YOLO

- YOLOv1 Изображението се разделя на мрежа (grid). Всяка клетка предсказва координати на обект и вероятност за обект
- YOLOv2 въвежда anchor boxes и подобрява точността
- YOLOv3 добавя multi-scale detection за по-добра детекция на малки обекти
- YOLOv4 / YOLOv5 по-добри архитектури и техники за обучение
- YOLOv6 / YOLOv7 оптимизация за реални приложения
 - разделяне на задачи (classification / localization)
- YOLOv8 anchor-free модел
 - по-модерна архитектура
 - поддържа повече задачи (сегментация, поза и др.)

Алгоритъм YOLO



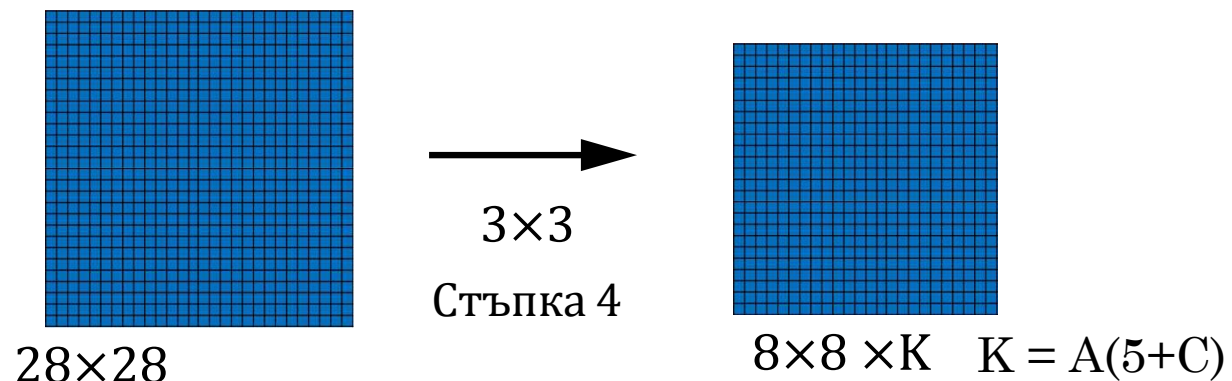
Target output
 $3 \times 3 \times 8$

Етикети за обучение за
всяка клетка от мрежата:

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix} + \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Откриване на обекти



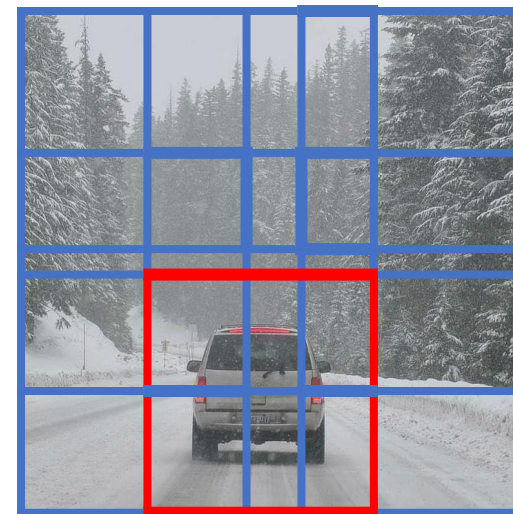
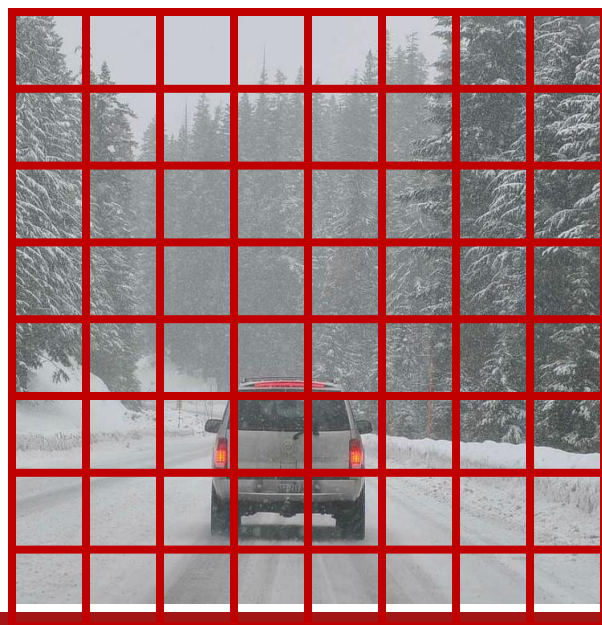
YOLO:

$$\text{filters} = A \times (5 + C)$$

където: A = брой anchor box

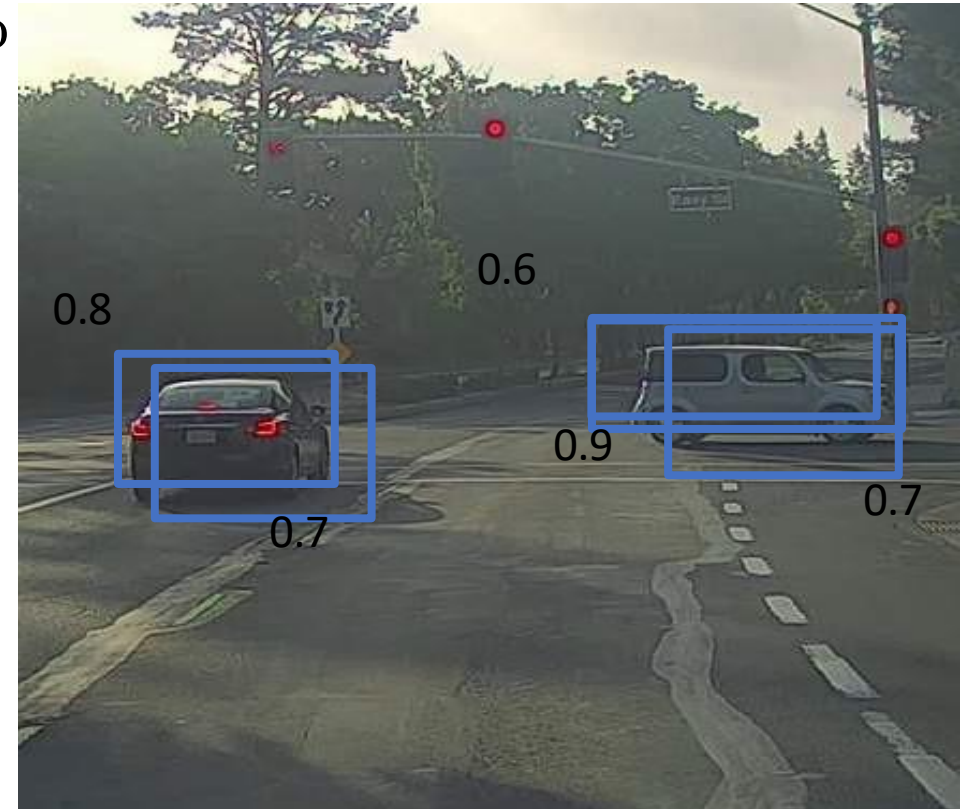
C = брой класове

$$(p_c + b_x + b_y + b_w + b_h = 5)$$



Non-max suppression algorithm

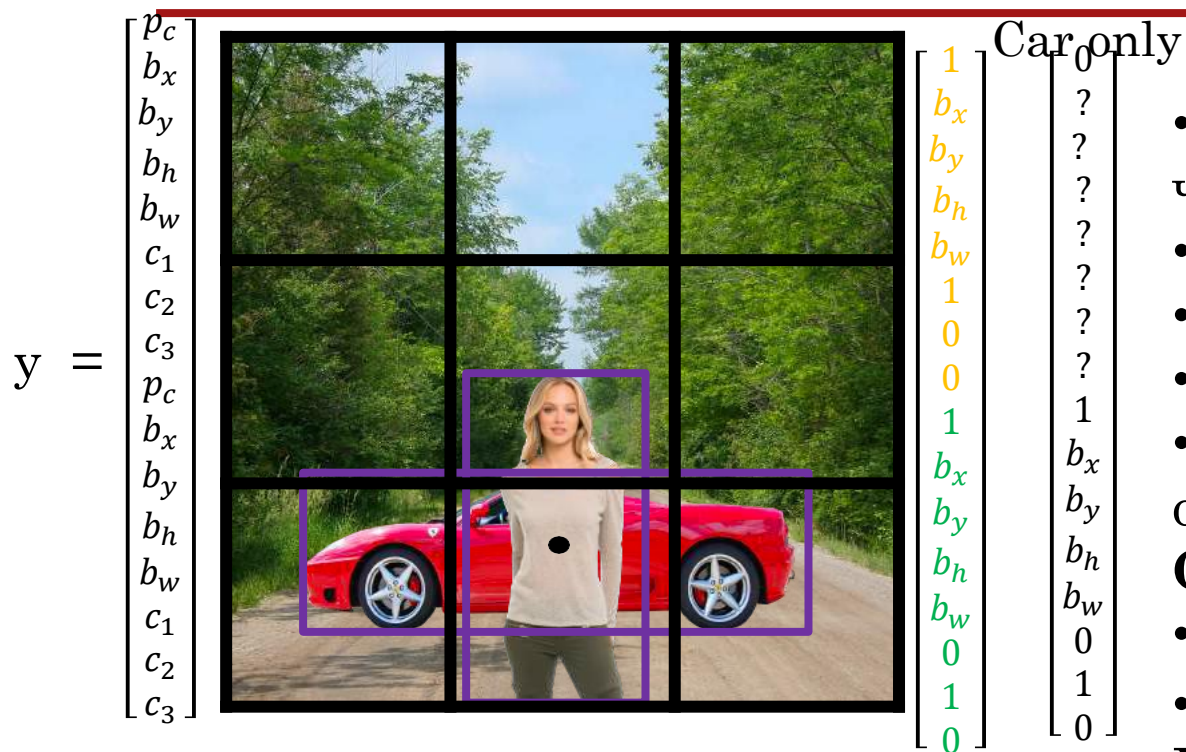
- Моделът връща множество рамки за един и същ обект (поради различни anchor-и), както и рамки за различни обекти.
- Необходимо е да се премахнат дублиращите се рамки (за един и същ обект), като се запазят тези, които отговарят на различни обекти.
- Non-Maximum Suppression (NMS) избира най-добрата рамка за даден обект и премахва останалите, които силно се припокриват с нея (т.е. отнасят се до същия обект), но запазва рамките, които са на различни позиции (различни обекти).



Откриване на обекти

Анкери

Анкерна кутия



- В една клетка може да има няколко обекта (напр. човек и кола)
- Всяка клетка има A anchor box-а с различни форми
- Всеки anchor може да предскаже един обект
- Обектът се присвоява на anchor с най-голям IoU
- В една клетка могат да се предскажат няколко обекта чрез различни anchor-и

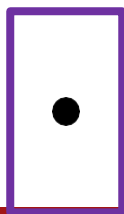
Откриване (YOLO):

- b_x, b_y — отместване спрямо клетката
- b_w, b_h — спрямо anchor box-а

Етикети (dataset):

- координатите са спрямо изображението

Anchor box 1:



Anchor box 2:



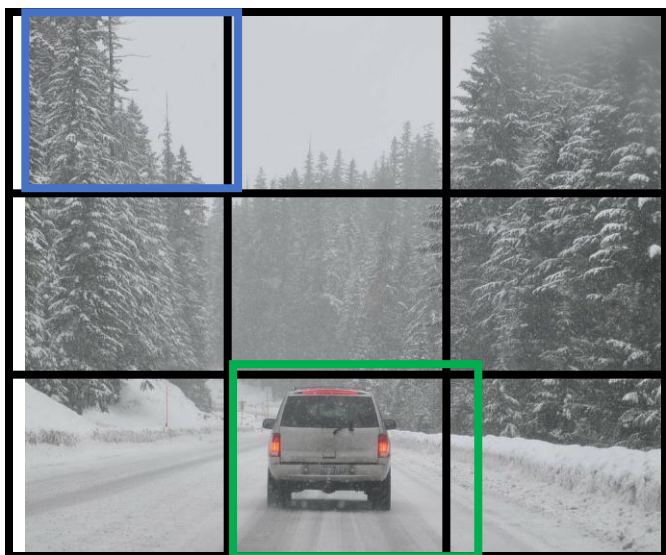
Откриване на обекти обучение

Модел

- Мрежата генерира решетка от характеристики (feature maps)
- Всяка клетка отговаря за обекти, чиито център попада в нея
- За всяка клетка и всеки anchor box моделът предсказва:
 - вероятност за наличие на обект (p_c)
 - координати на ограничаващата рамка b_x, b_y, b_w, b_h
 - вероятности за класове
- Така се генерират множество кандидат-рамки за различни обекти
- Накрая се прилага **Non-Max Suppression**, за да се премахнат припокриващите се предсказания

Обучение

- По време на обучението знаем реалните обекти. Той се присвоява на една клетка и един anchor box. За него задаваме $p_c = 1$, координатите на рамката и класа. За всички останали клетки и anchor-и задаваме $p_c = 0$ или се игнорират



- 1 - pedestrian
2 - car
3 - motorcycle

$y =$

$$\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix} \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix}$$

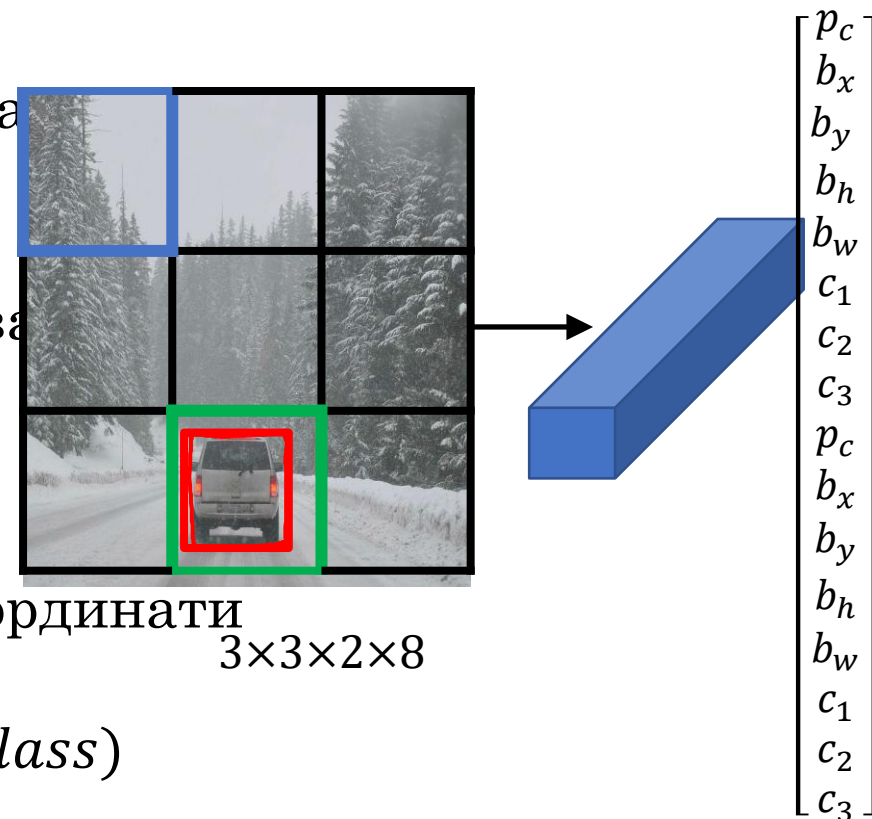
$$Y = 3 \times 3 \times 2 \times 8$$

Anchor boxes

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Приложение

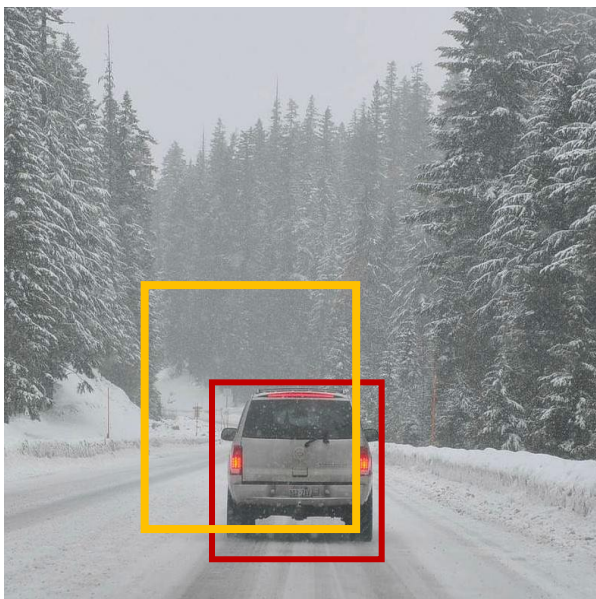
- Ново изображение се подава към обучената мрежа
- Мрежата генерира решетка от предсказания (maps)
- За всяка клетка и всеки анкер моделът предсказва
 - вероятност за наличие на обект (p_c)
 - координати на рамката (b_x, b_y, b_w, b_h)
 - вероятности за класове
- Предсказанията се преобразуват в реалникоординати (спрямо анкерите)
- Изчислява се score за всяка рамка: $score = p_c \cdot P(class)$
- Генерират се множество кандидат-рамки
- Прилага се **Non-Max Suppression (NMS)**, за да се изберат крайните обекти



Откриване на обекти

post-processing

Оценка на локализацията на обектите

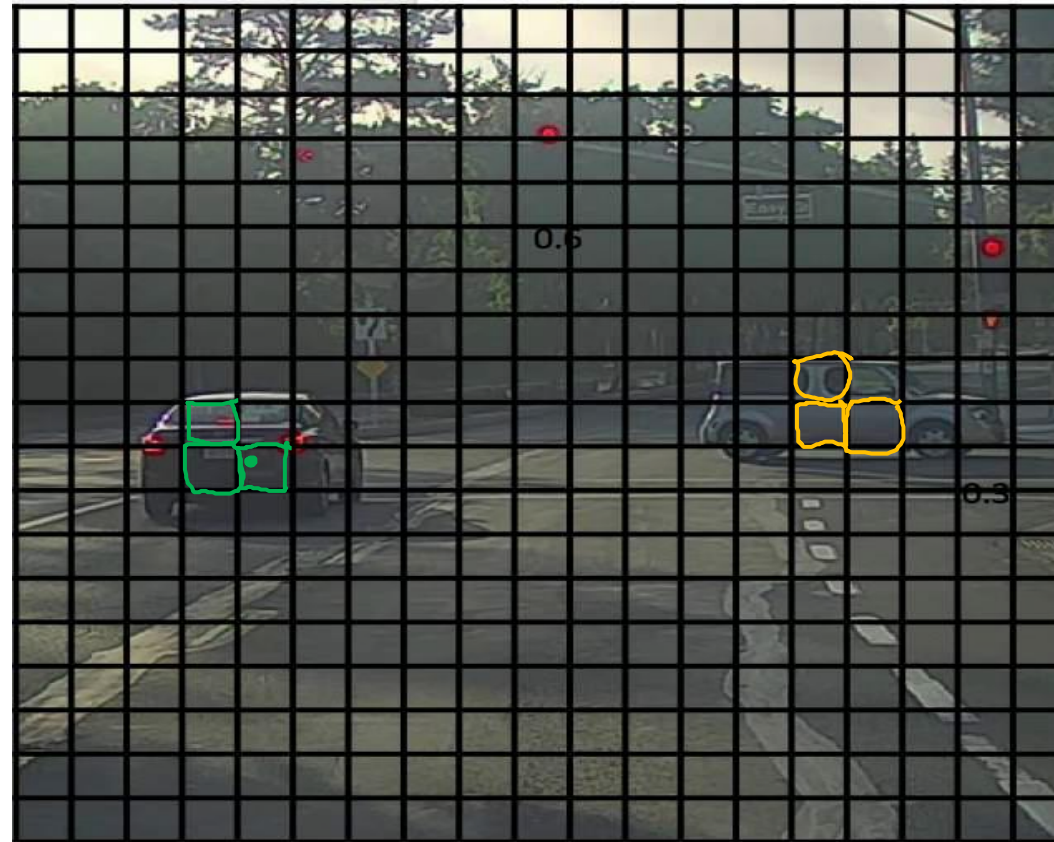


$$IoU = \frac{intersection}{union}$$

“Correct” if $IoU \geq 0.5$

IoU е мярка за припокриване между две ограничителни рамки

Non-max suppression algorithm



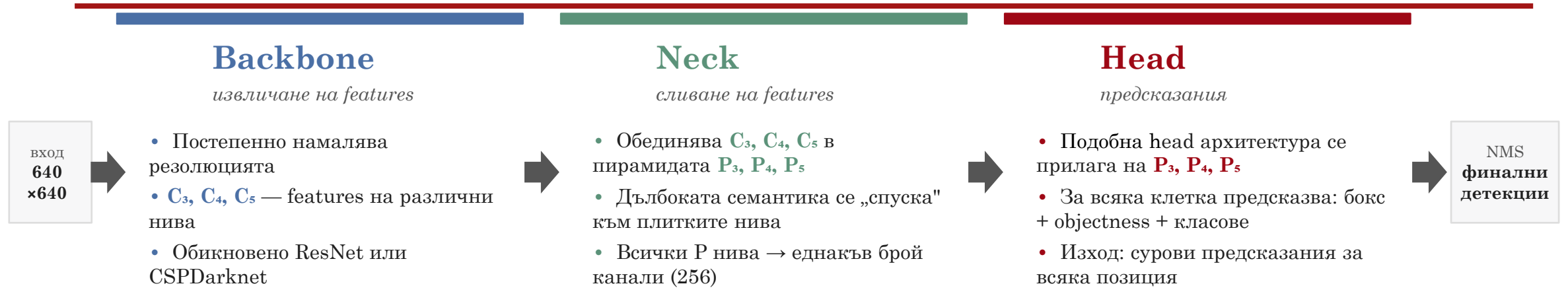
Премахване на припокриващи се рамки (NMS)

- Премахване на припокриващи се рамки
- Премахват се всички рамки с ниска увереност.
След това алгоритъмът може да се приложи по два начина: по класове или глобално
 - рамките се сортират по вероятност p_c в низходящ ред;
 - докато има останали рамки:
 - избира се рамката с най-висока стойност на p_c ;
 - добавя се към резултата;
 - премахват се всички рамки с $\text{IoU} > \text{threshold}$ спрямо нея.

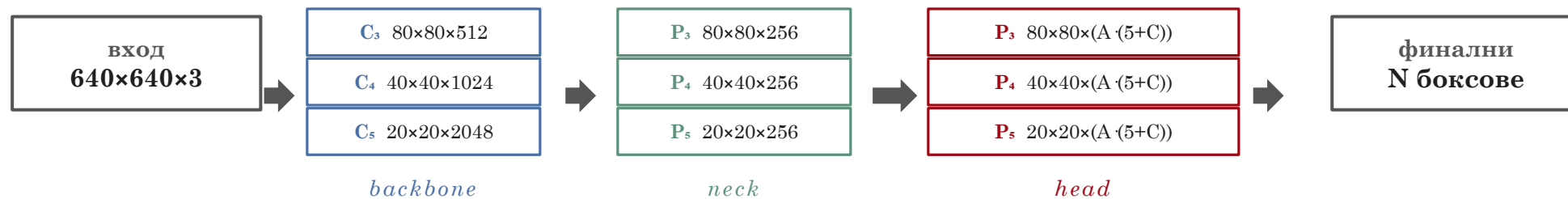
Основни YOLO архитектури

- Съвременните YOLO модели имат 3 части:
 - Backbone извлича характеристики (features)
 - ResNet/Darknet → C3: $80 \times 80 \times 256$, C4: $40 \times 40 \times 512$, C5: $20 \times 20 \times 1024$
 - Neck: комбинира информация от различни мащаби
 - карти на характеристики в различни мащаби
 - Head прави крайните предсказания:
 - рамки
 - класове
 - вероятност за обект

YOLO архитектура три основни блока



Размерност на тензорите



Ефективният stride определя разделянето на изображението на решетка

- Чрез слоеве със $\text{stride} > 1$ основната мрежа намалява резолюцията и получената feature карта формира решетка

брой клетки = *размер на изображението / stride*

За изображение 640×640 px

ниво	stride	решетка	px / клетка
P_3	8	80×80	8×8
P_4	16	40×40	16×16
P_5	32	20×20	32×32

Какво представлява една клетка

Клетка (i, j) отговаря за квадратен регион $\text{stride} \times \text{stride}$ пиксела от изображението.

Център в пиксели: $((i + 0.5) \cdot \text{stride}, (j + 0.5) \cdot \text{stride})$

Backbone-ът намалява резолюцията на всеки stage чрез stride-2 конволюция. След N стъпки общият stride е 2^N .

Конкретен пример · колата

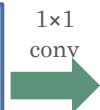
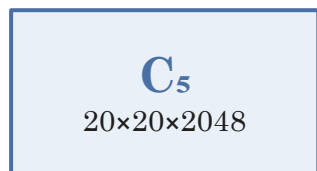
Център на gt бокса в пиксели: $(0.5 \times 640, 0.7 \times 640) = (320, 448)$

На P_4 (stride = 16):

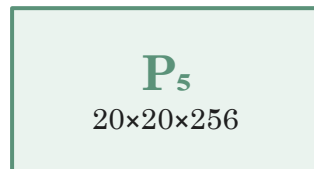
$i = \lfloor 320 / 16 \rfloor = 20$ · $j = \lfloor 448 / 16 \rfloor = 28 \rightarrow$ клетка $(20, 28)$

клетката отговаря за регион от $\text{stride} \times \text{stride}$ px, но 'вижда' много повече заради дълбочината.

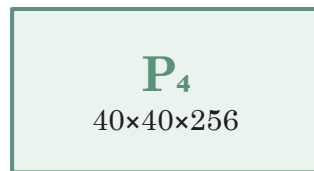
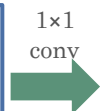
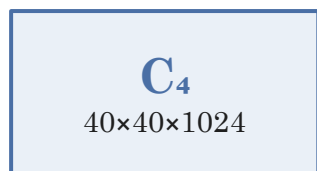
BACKBONE



NECK (FPN)

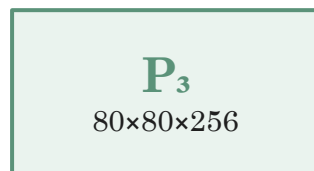
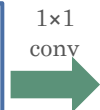
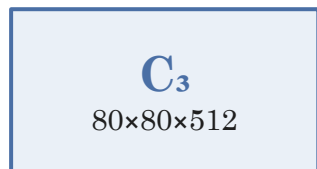


upsample ×2 + add



*top-down
path*

upsample ×2 + add



Top-down път — започва от **C₅**, upsample-ва и добавя към лявото ниво.

1×1 conv изравнява броя канали (напр 256).

P₃, P₄, P₅ имат пространствената точност на C-овете и семантиката на C₅.

плитките нива виждат добре къде, но не разбират *какво* е обектът.

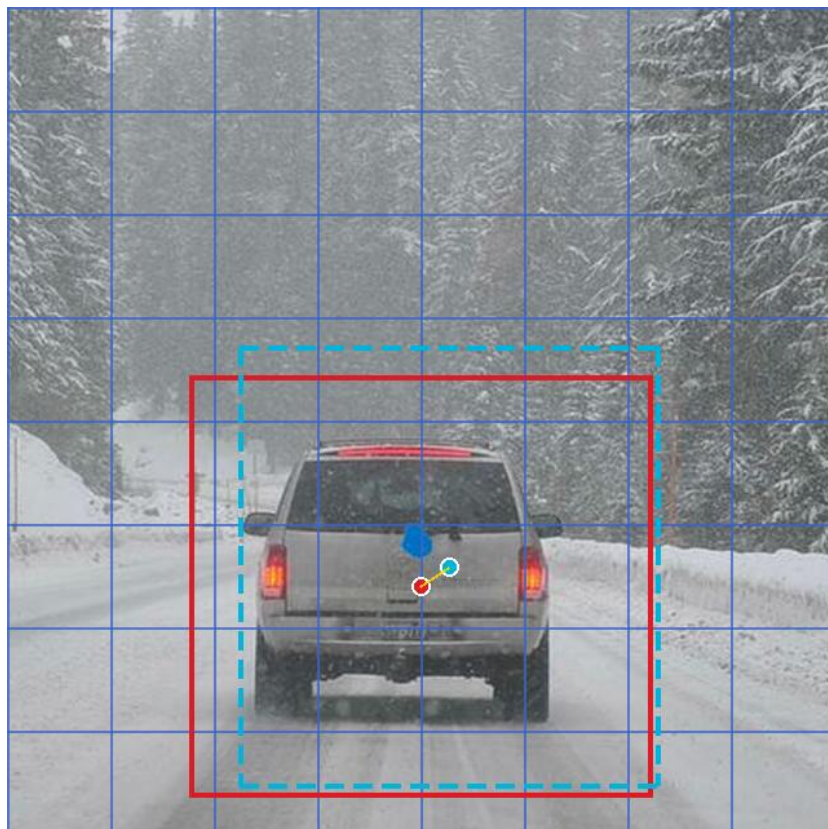
всяко P ниво има и точна локализация, и богата семантика.

P₃ 80×80, stride 8
малки обекти

P₄ 40×40, stride 16
средни обекти

P₅ 20×20, stride 32
големи обекти

Етикет и предсказание Сравнение на рамките



Истинска рамка b (от етикета)

Предсказана рамка $\hat{b}$

ЕТИКЕТ · ground truth от файла

$$b = (b_x, b_y, b_w, b_h)$$

за колата: (0.5, 0.7, 0.55, 0.5) · нормализирани координати

ПРЕДСКАЗАНИЕ · декодирано от изхода на главата

$$\hat{b}_x = (c_x + \sigma(t_x)) / S \quad \hat{b}_w = p_w \cdot \exp(t_w)$$

$$\hat{b}_y = (c_y + \sigma(t_y)) / S \quad \hat{b}_h = p_h \cdot \exp(t_h)$$

S = размер на grid-а (напр. 80)

t_x, t_y, t_w, t_h са сурови изходи на главата за позитивната клетка

Сравнение · загубата за бокса

$$L_{box} = 1 - CIoU(\hat{b}, b)$$

CIoU отчита припокриване, разстояние между центровете и съотношение на страните.

Перфектно припокриване $\rightarrow loss = 0$ · няма припокриване $\rightarrow loss \rightarrow 1+$

Пример Откриване (detection head)

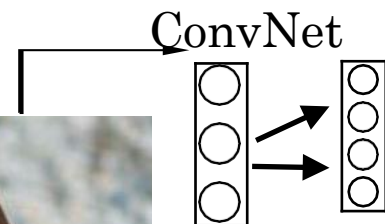
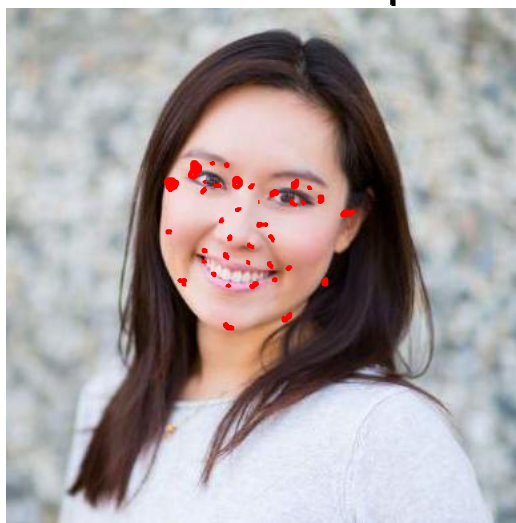


Откриване на обекти

Откриване на характерни точки

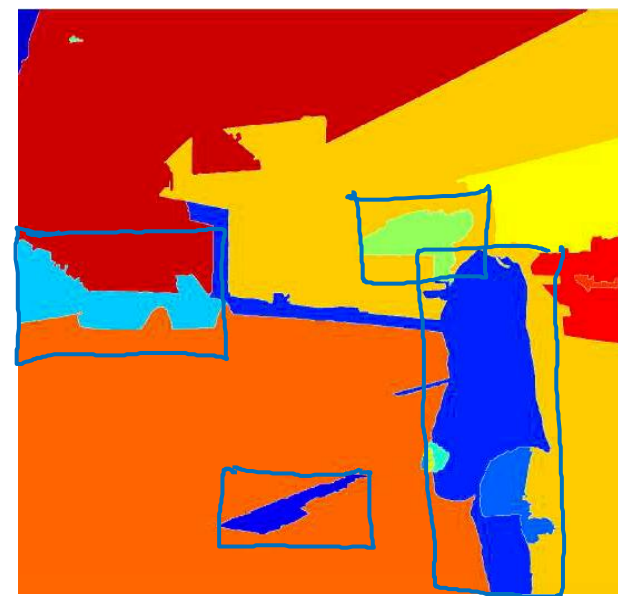
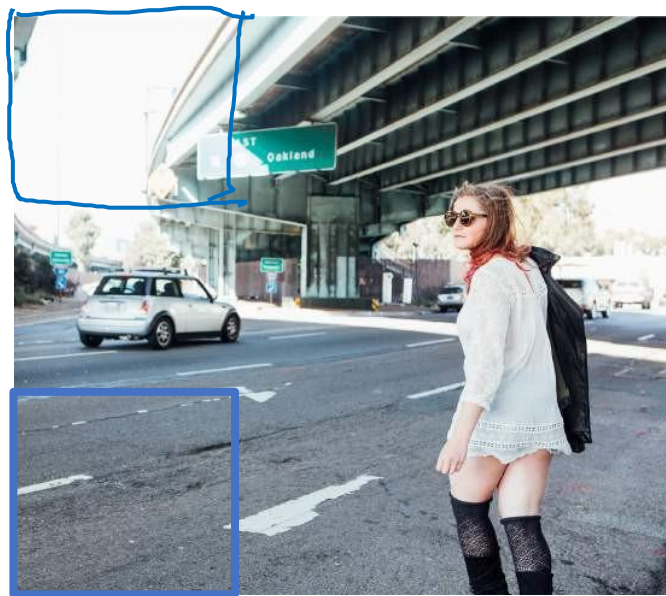
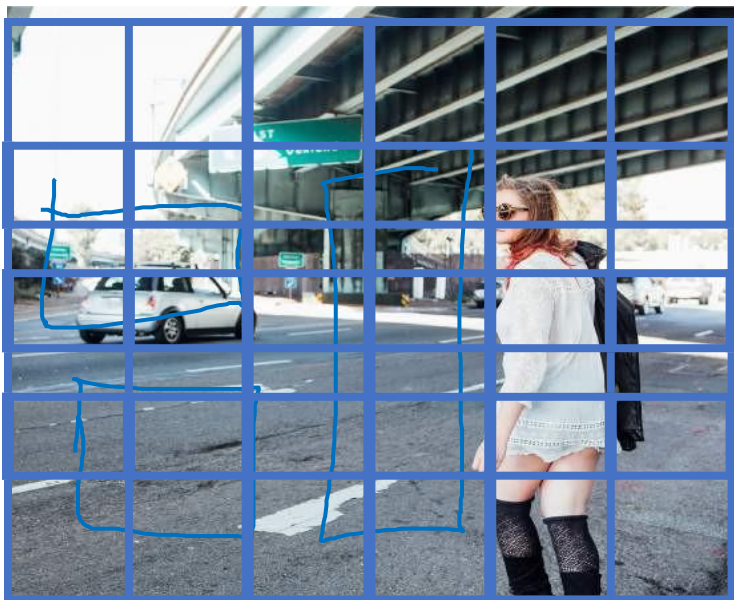
Разпознаване на характерни точки

- Рамката има 4 точки, но те нямат семантично значение — просто са ъгли на правоъгълник около обекта.
- Характерните точки имат фиксирано семантично значение — точка 1 = "ляв ъгъл на устата", точка 2 = "връх на носа" и т.н.



Предложение за регион: R-CNN

- Много кандидат-региона (region proposals)
- Избрани региони
- Сегментация / групиране (selective search)



[Girshik et. al, 2013, Rich feature hierarchies for accurate object detection and semantic segmentation]

Faster algorithms

- R-CNN: Предлагат се региони. Класифицират се предложените региони един по един. Изходен етикет + рамка.
- Fast R-CNN: Използва се конволюционна реализация на плъзгащи се прозорци, за да класифицират всички предложени региони.
- Faster R-CNN: Използва се конволюционна мрежа за предлагане на региони

[Girshik et. al, 2013. Rich feature hierarchies for accurate object detection and semantic segmentation]

[Girshik, 2015. Fast R-CNN]

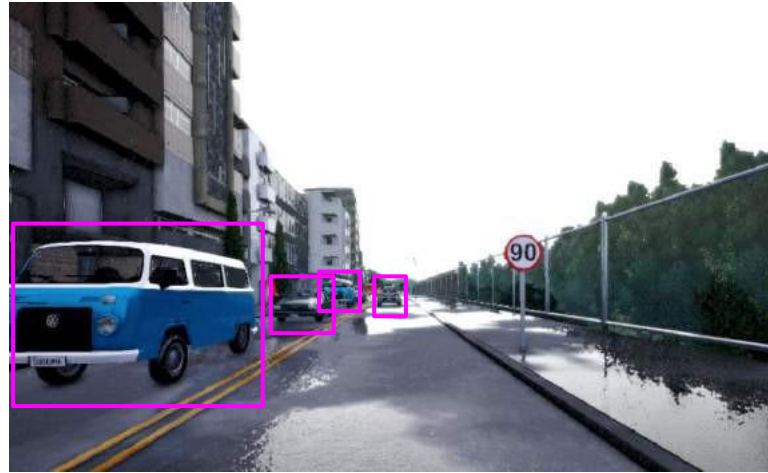
[Ren et. al, 2016. Faster R-CNN: Towards real-time object detection with region proposal networks]

Семантична сегментация с U-Net

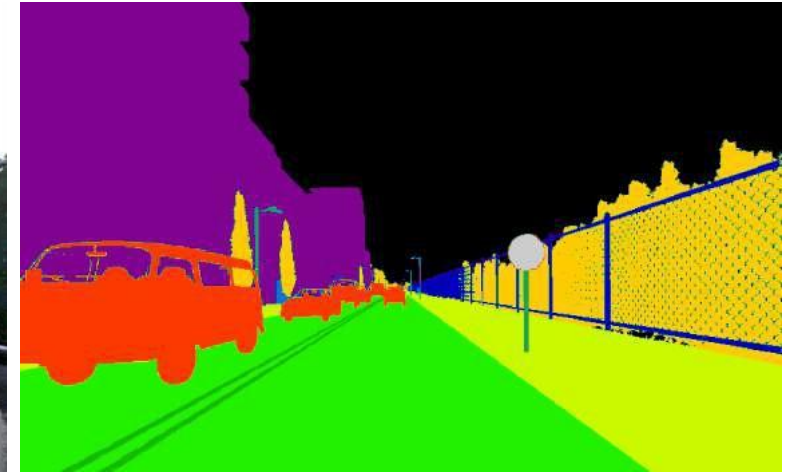
Object Detection vs. Semantic Segmentation



Input image

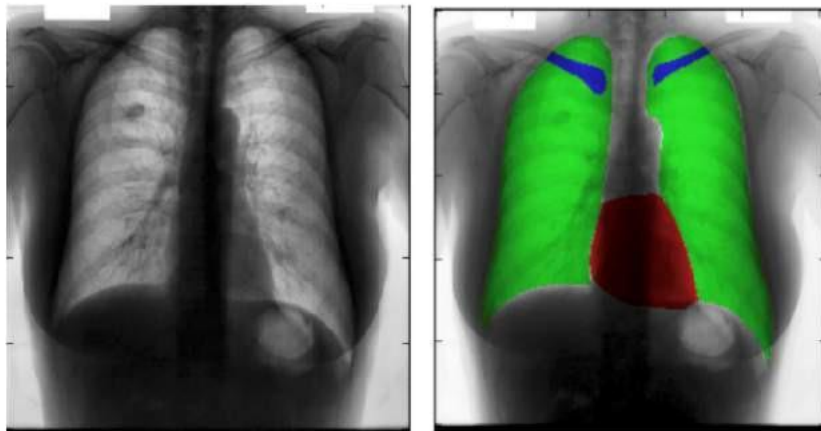


Object Detection

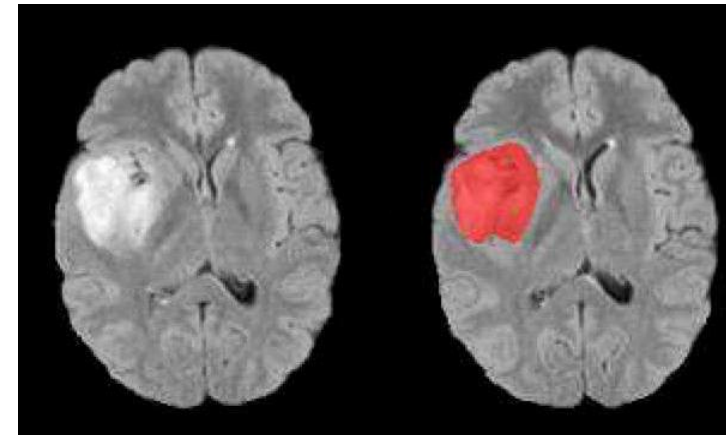


Semantic Segmentation

Motivation for U-Net



Chest X-Ray

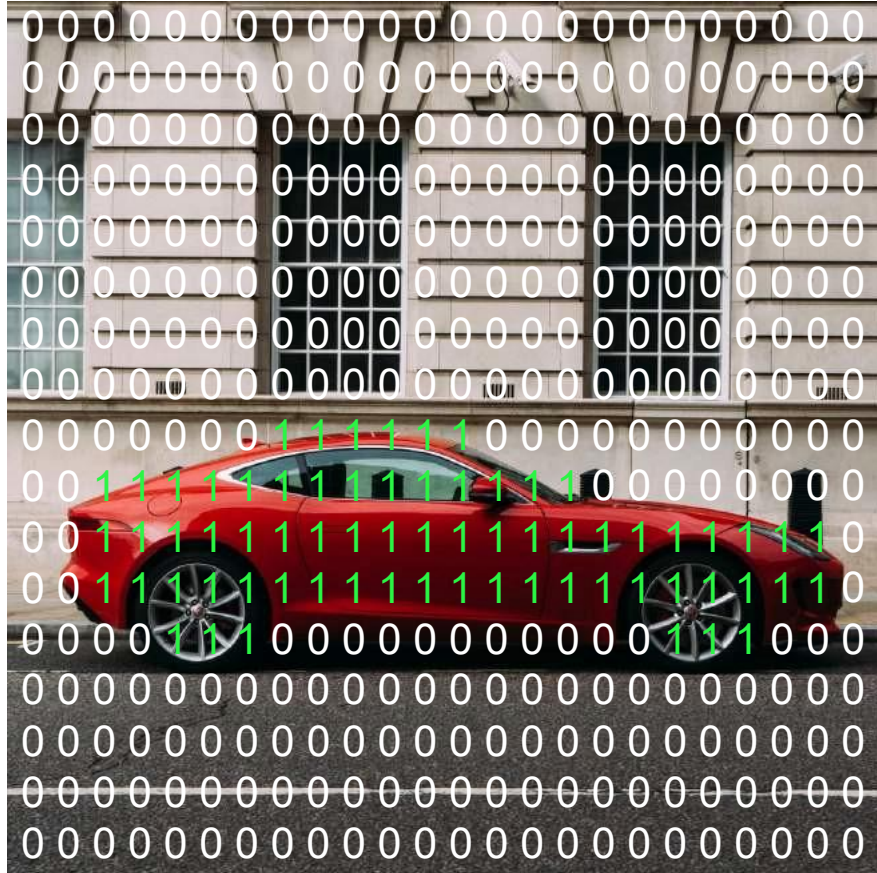


Brain MRI

[Novikov et al., 2017, Fully Convolutional Architectures for Multi-Class Segmentation in Chest Radiographs]

[Dong et al., 2017, Automatic Brain Tumor Detection and Segmentation Using U-Net Based Fully Convolutional Networks]

Per-pixel class labels

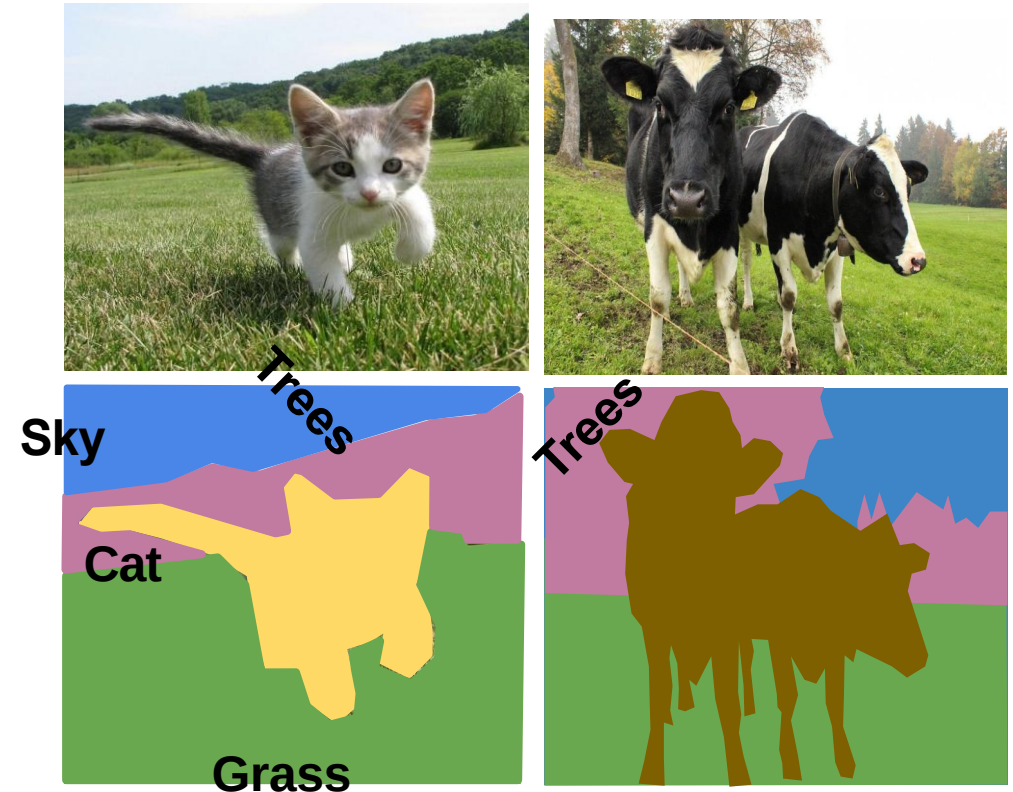


1. Car
0. Not Car

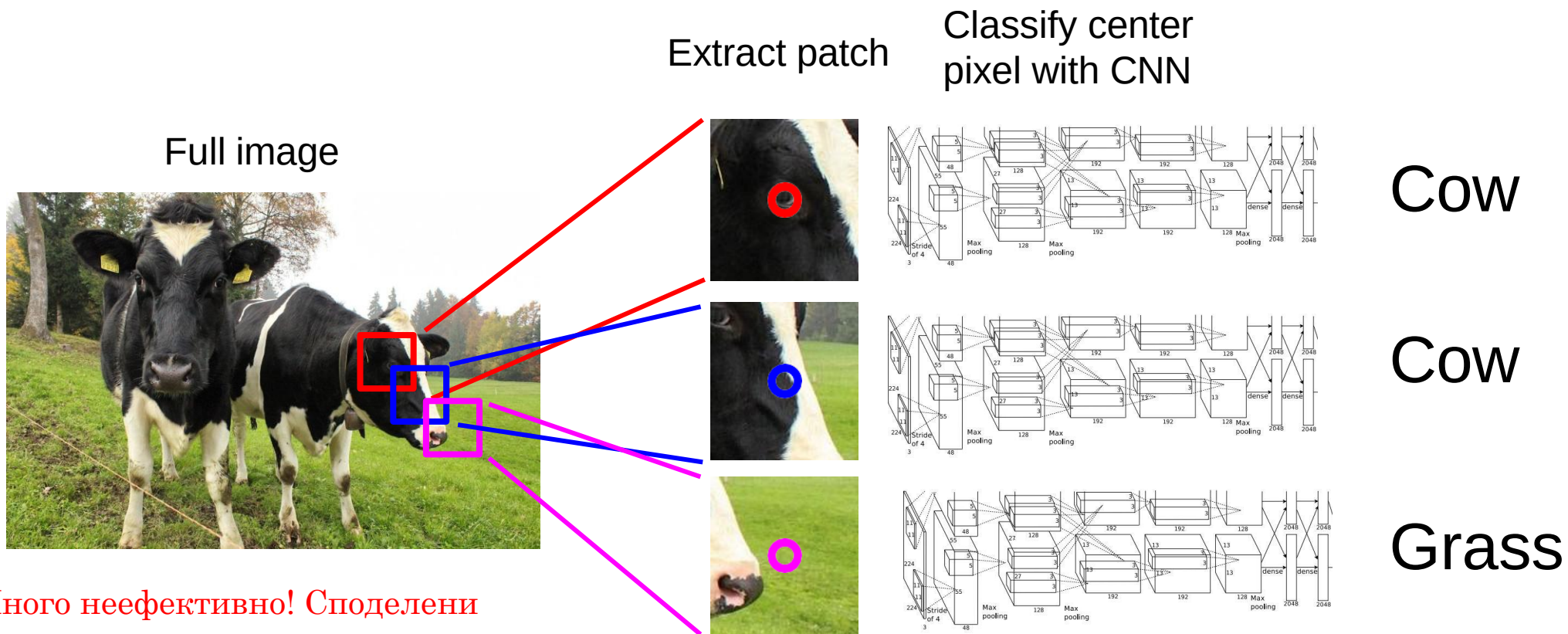
Семантична сегментация

- Всеки пиксел в изображението се етикетира с клас (категория)
- Не се разграничават отделните инстанции — интерес представлява само класът на всеки пиксел
- сегментация на отделни обекти

[This image is CC0 public domain](#)



Сегментация чрез плъзгащ се прозорец



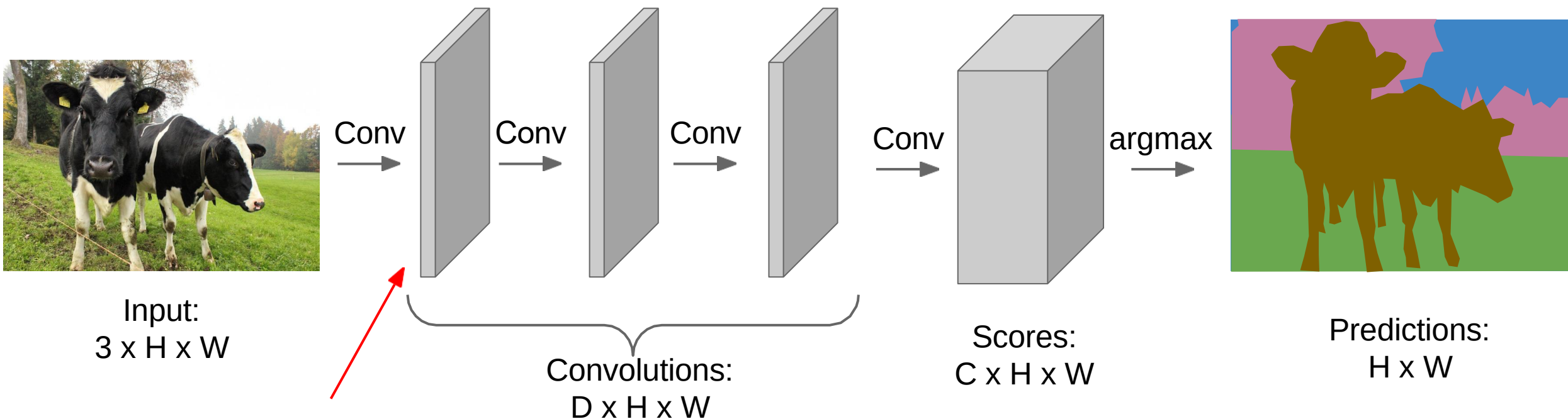
Много неефективно! Споделени
характеристики се използват
повторно между припокриващи се
пачове

Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013

Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

Напълно конволюционна

- Проектира се мрежа от конволюционни слоеве, без свиване, за да прогнозира за пикселите наведнъж!



конволюциите при оригинална резолюция на изображението ще бъдат много скъпи ...

Напълно конволюционна

• Down sampling и up sampling в мрежата!

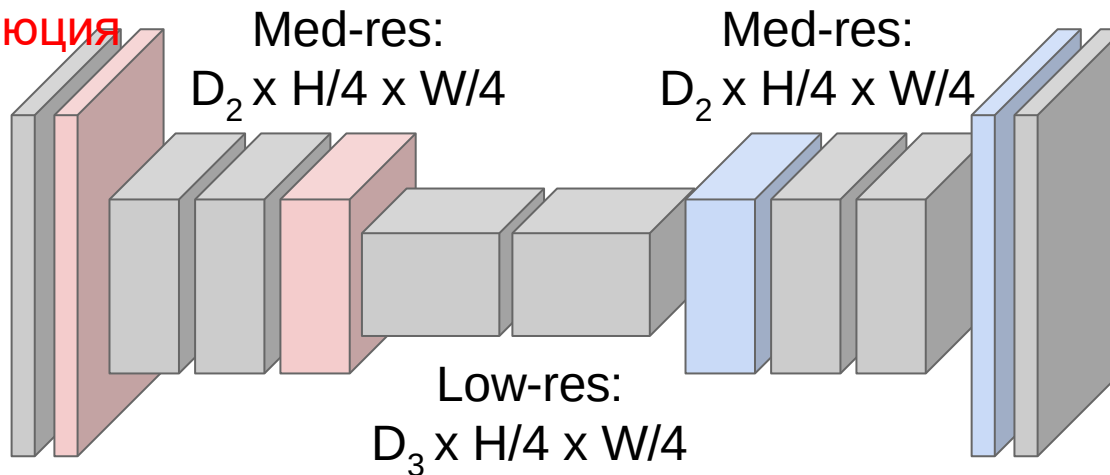
Down sampling:

Pooling, стъпкова конволюция



Input:
 $3 \times H \times W$

High-res:
 $D_1 \times H/2 \times W/2$



High-res:
 $D_1 \times H/2 \times W/2$

Upsampling:
???



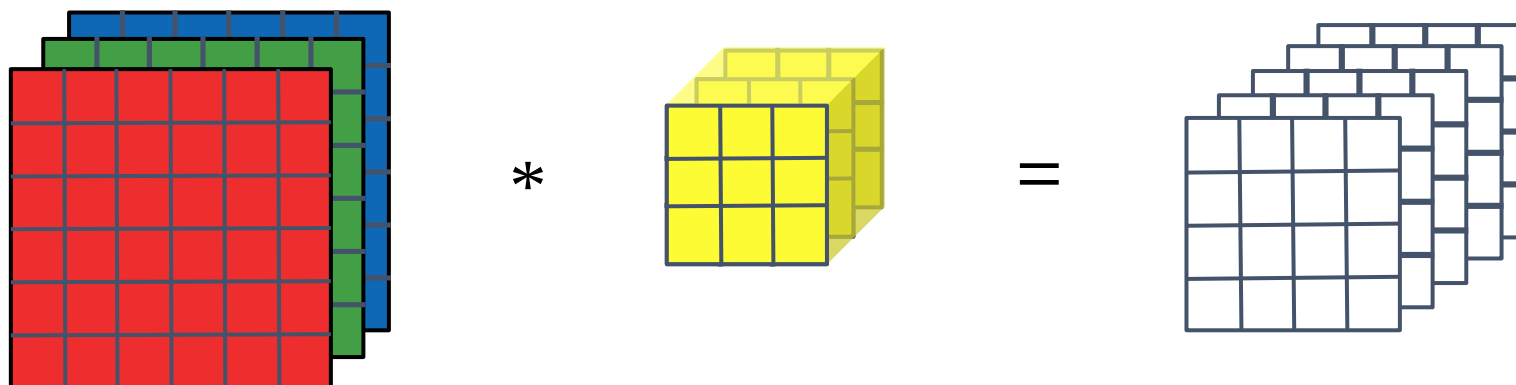
Predictions:
 $H \times W$

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015

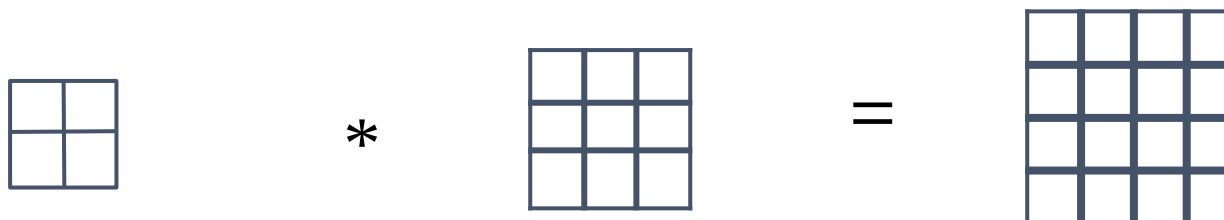
Noh et al. "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Транспонирана конволюция

Normal Convolution

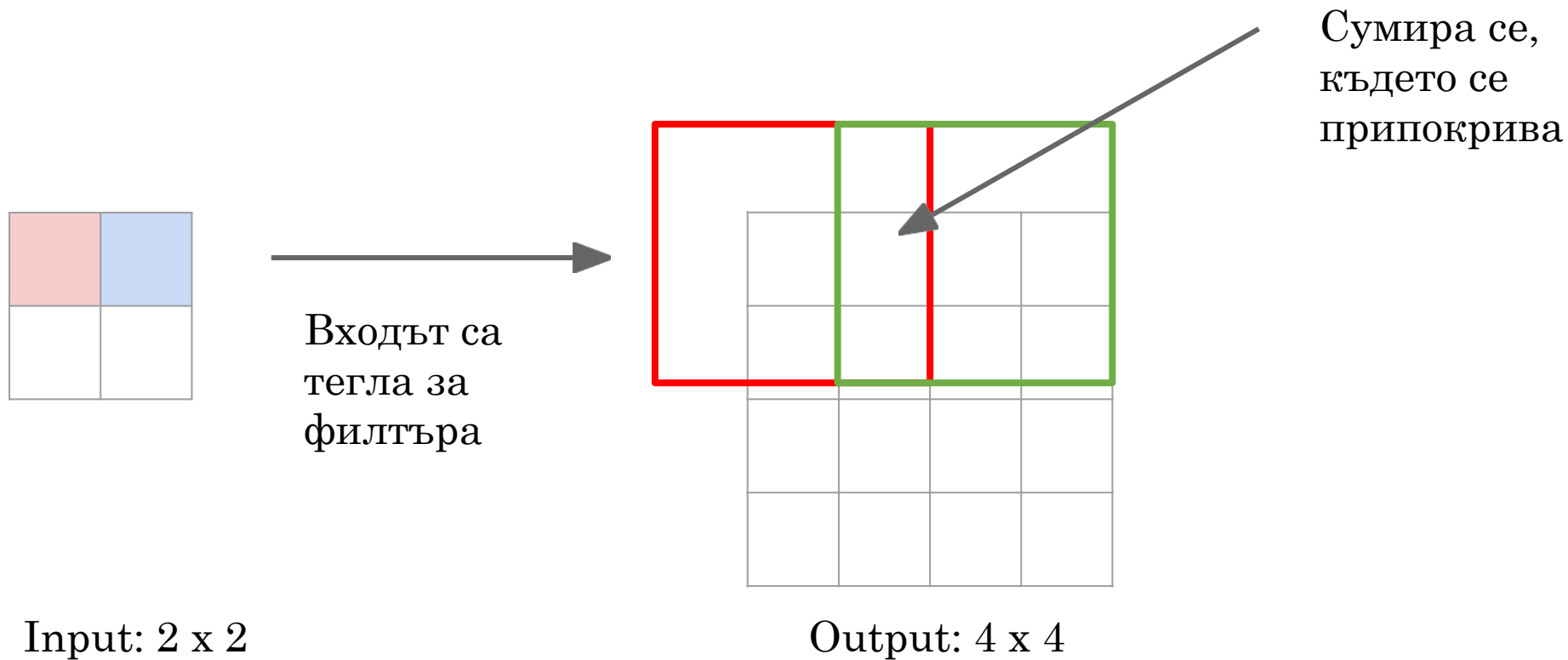


Transpose Convolution

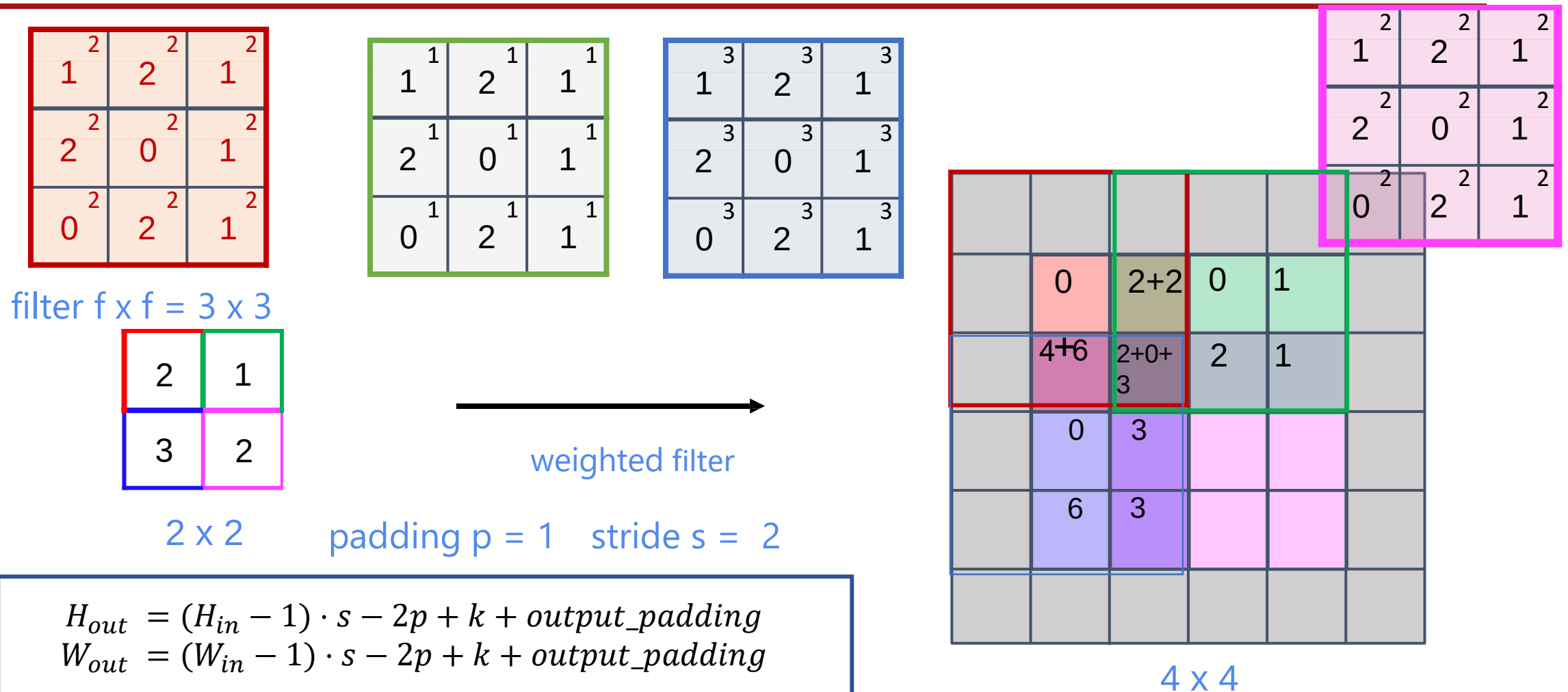


Транспонирана конволюция(upsampling с обучаеми филтри)

3 x 3 транспонирана конволюция, стъпка 2, пад 1



Транспонирана конволюция



Up sampling в мрежата: "Unpooling"

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

"Bed of Nails"

1	2
3	4



1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input: 2 x 2

Output: 4 x 4

Up sampling в мрежата: “Max Unpooling”

Max Pooling

Запомня се кой елемент е максимален!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8



5	6
7	8



Останалата част
от мрежата

Вход: 4 x 4

Output: 2 x 2

Съответстващи двойки слоеве за
понижаване и повишаване на
семплирането

Max Unpooling

Използват се позиции
от слоя за пулинг

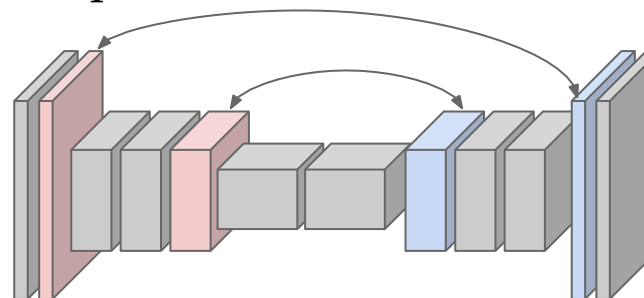
1	2
3	4



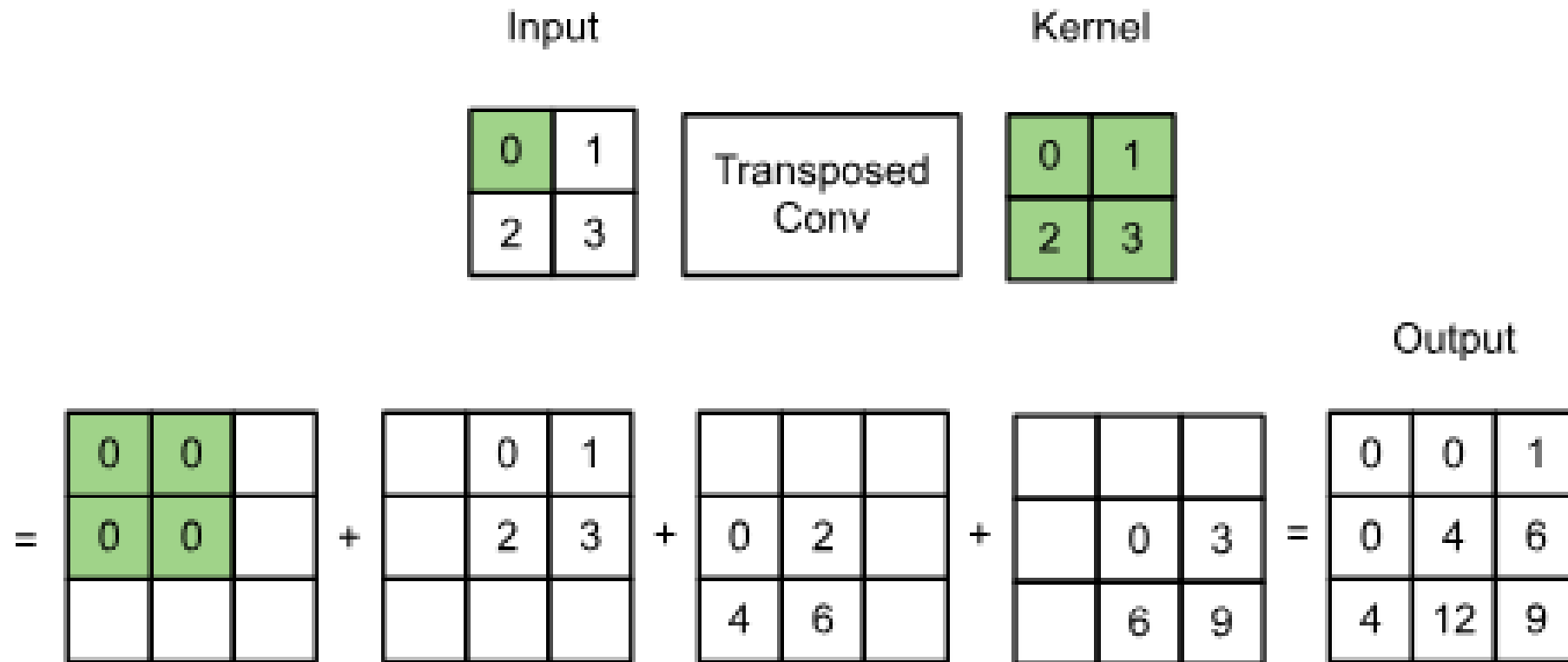
0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

Input: 2 x 2

Output: 4 x 4

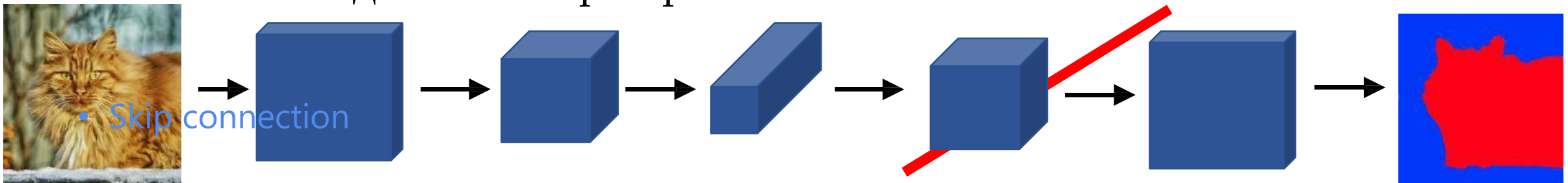


Пример

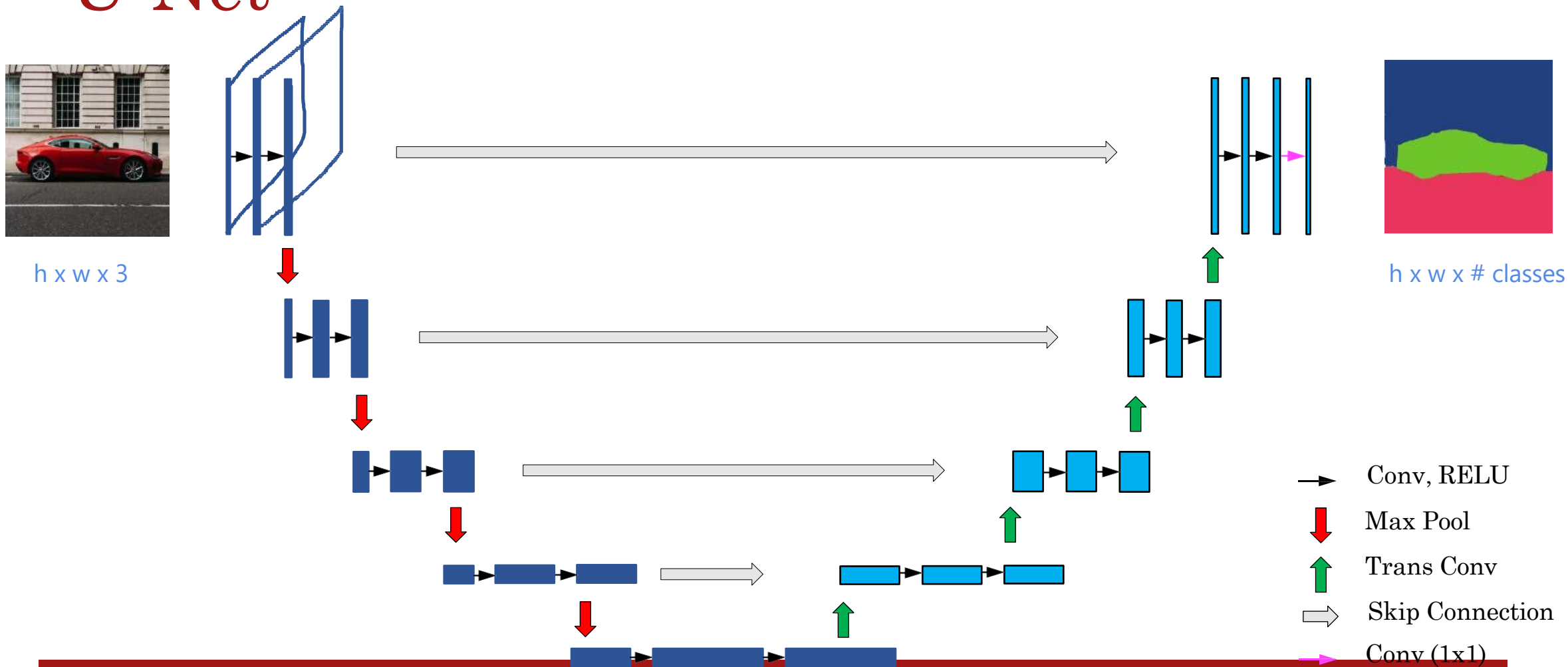


Дълбоко самообучение за семантична сегментация

- Encoder (лява част):
 - извлича характеристики чрез convolution и pooling
 - намалява пространствената резолюция
- Decoder (дясна част):
 - възстановява резолюцията чрез upsampling
 - предсказва пикселни класове
- Skip connections:
 - свързват encoder и decoder
 - запазват детайлна пространствена информация

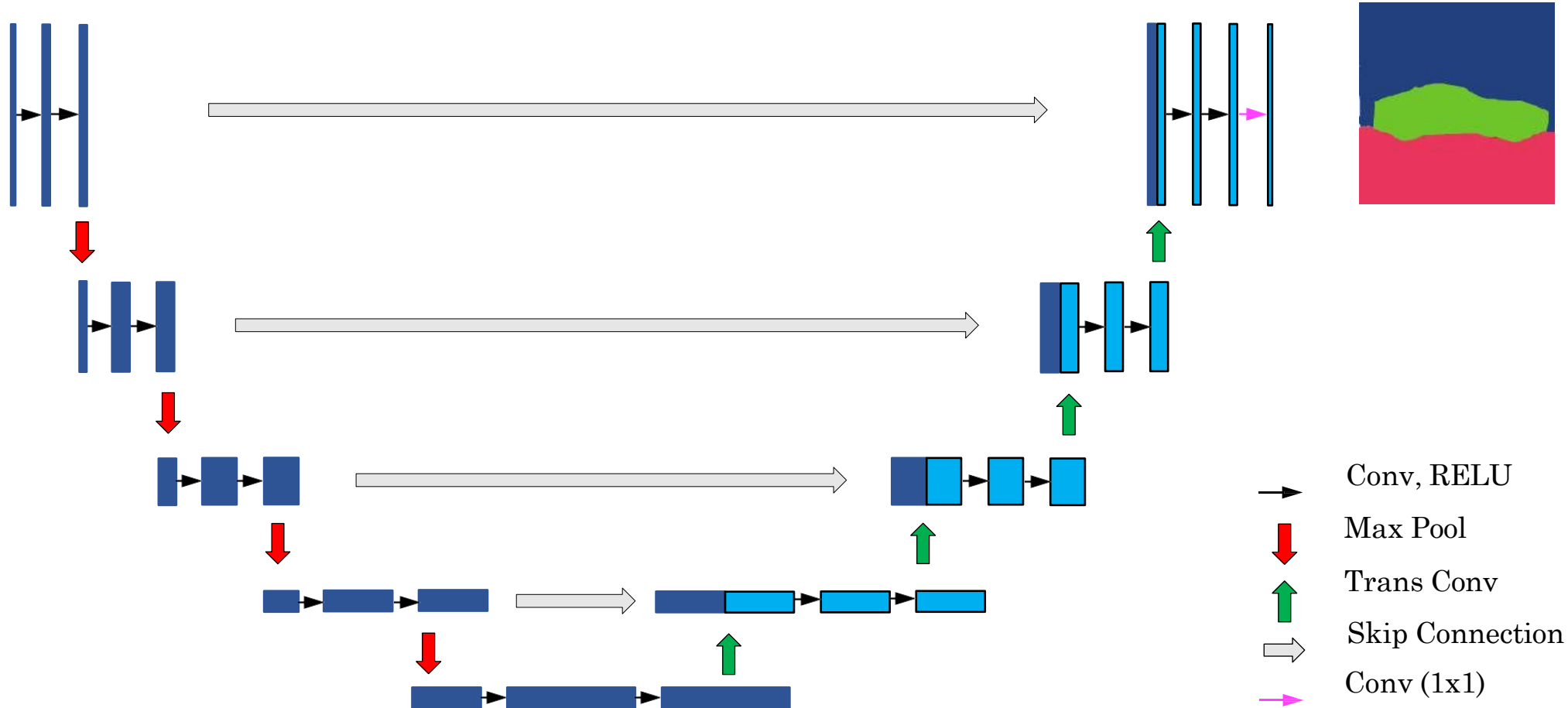


-U-Net



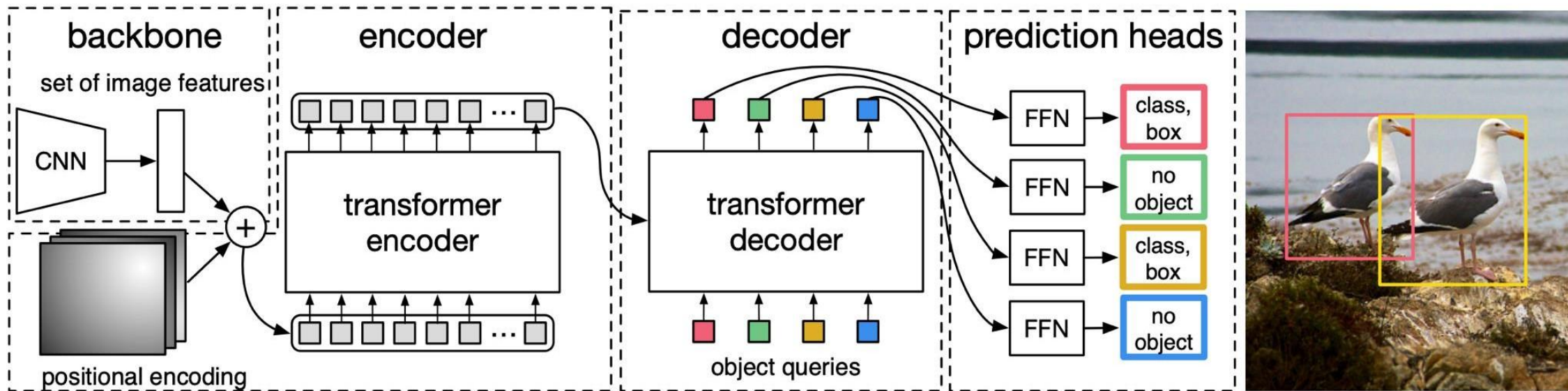
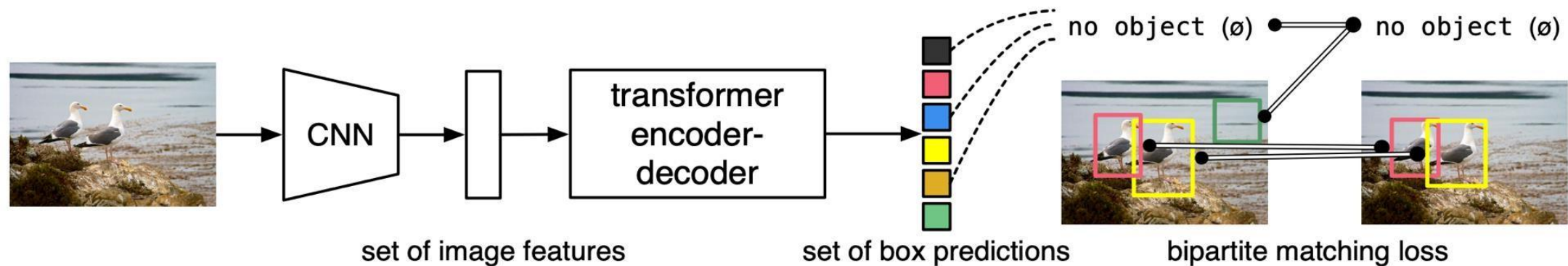
[Ronneberger et al., 2015, U-Net: Convolutional Networks for Biomedical Image Segmentation]

-U-Net



[Ronneberger et al., 2015, U-Net: Convolutional Networks for Biomedical Image Segmentation]

Откриване на обекти с трансформатори: DETR



Reference

1. deeplearning.ai Andrew NG
2. CS231n: Convolutional Neural Networks for Visual Recognition
3. Deep Learning. Ian Goodfellow, Yoshua Bengio, and Aaron Courville